

Lucrare de laborator : 4

Construirea si utilizarea serviciilor in .NET

1. Introducere

Conectivitatea între aplicații este foarte importantă. Conectivitatea este, în orice caz, foarte importantă pentru acest lucru, dar este deosebit de importantă între aplicații. Conectarea aplicației web a fost o mare provocare înainte de apariția unor tehnologii precum SOAP (Simple Object Access Protocol). Motivul pentru care a fost atât de dificil a fost că au existat atât de multe tehnologii în care oamenii lucrau. Aplicațiile au fost găzduite pe diferite tipuri de servere etc. Dar aceste lucruri nu ar trebui să fie o barieră pentru a facilita comunicarea între aplicații. Singura cerință a fost de a avea anumite standarde de urmat și modalități standard de a face lucrurile astfel încât aplicațiile să devină capabile să comunice cu alte aplicații, indiferent de tehnologiile utilizate.

1.1. SOAP si Servicii Web

SOAP și XML au creat soluția pentru problema cu care se confruntau dezvoltatorii înainte. SOAP este un protocol standard bazat pe XML care comunica prin HTTP. Ne putem gândi la SOAP ca format de mesaj pentru trimiterea de mesaje între aplicații folosind XML. Este independent de tehnologie, platformă și este extensibilă.

Avem SOAP și XML pentru a ne oferi conectivitate între aplicații. Înseamnă că trebuie să scriu singur lucruri specifice XML și SOAP pentru a facilita această comunicare?

Unde apar serviciile Web în imagine? Ei bine, serviciile Web sunt mecanismul pe care sistemul ASP.NET îl oferă pentru a ne facilita scrierea de coduri pentru a facilita conectivitatea între aplicații. Ca dezvoltator ASP.NET, dacă am nevoie de o aplicație care va fi folosită de multe alte aplicații, atunci pot decide pur și simplu să scriu un serviciu web pentru el și cadrul ASP.NET va avea grijă de a face SOAP de nivel scăzut și de lucru XML pentru noi .

Cealaltă parte a monedei este, dacă aplicația noastră ASP.NET dorește să utilizeze un serviciu web, adică o aplicație care îmi oferă interfață bazată pe SOAP pentru comunicare. Deci, ceea ce vom face acum este să scrieți un mic serviciu Web pentru a vedea cum putem avea aplicația noastră de comunicare gata pentru alte aplicații și, în al doilea rând, vom încerca să consumăm un webservice pentru a înțelege modul în care putem folosi alte aplicații din aplicația noastră.

1.2. SOAP (Simple Object Access Protocol Overview)

Protocolul de acces simplu de obiect (SOAP) a fost creat în 1998 de Dave Winer, Don Box, Bob Atkinson și Mohsen Al-Ghosein cu sprijinul Microsoft. SOAP este un cadru de mesagerie care a câștigat un suport pe scară largă în comunitățile Java, .NET și open source în perioada începută a anilor 2000. Acesta a servit ca bază a multor proiecte de servicii Web și oferă mecanismul prin care multe alte standarde de servicii Web comunică.

SOAP (Simple Object Access Protocol) este o modalitate pentru un program care rulează într-un fel de sistem de operare (cum ar fi Windows 2000) de a comunica cu un program în același sau alt tip de sistem de operare (cum ar fi Linux) Wide Web Hypertext Transfer Protocol (HTTP) și limbajul extensibil de marcare (XML) ca mecanisme pentru schimbul de informații.

Deoarece protocoalele Web sunt instalate și disponibile pentru utilizarea de către toate platformele majore de sistem de operare, HTTP și XML oferă o soluție deja la îndemână pentru problema modului în care programele care rulează în cadrul diferitelor sisteme de operare dintr-o rețea pot comunica între ele. SOAP specifică exact modul de codare a unui antet HTTP și a unui

fișier XML, astfel încât un program dintr-un singur computer să poată apela un program pe alt computer și să îi transmită informații. De asemenea, specifică modul în care programul numit poate returna un răspuns.

Ed Tittel, scriitor cu normă întreagă și trainer XML, a scris un articol despre SOAP 1.2. El afirmă că în aprilie 2007, Consorțiul World Wide Web a lansat o colecție de patru documente ca recomandări W3C. Împreună, aceste documente consacră o SOAP 1.2 Second Edition drept cea mai recentă și cea mai mare definiție a informațiilor bazate pe XML, potrivite pentru schimbul de informații structurate și tipizate între colegii într-un mediu de execuție distribuit și descentralizat.

Desigur, SOAP este folosit de ceva timp. Recomandările originale SOAP 1.2 au fost lansate în 2003. Ediția a doua a documentelor SOAP de bază este interesant de văzut, deoarece W3C a făcut mari eforturi pentru a se asigura că schimbările sunt ușor de văzut (adăugirile apar în verde, modificările în galben și eliminări în roșu spălat). Potrivit organizației, a doua ediție a SOAP 1.2 "încorporează corecții pentru erorile cunoscute" în prima ediție.

Există mai multe argumente între aceștia pentru utilizarea SOAP și cei opuși. Iată o listă de avantaje:

- Utilizarea SOAP prin HTTP permite o comunicare mai ușoară prin intermediul proxy-urilor și firewall-urilor decât tehnologia anterioară de execuție la distanță.
- SOAP este suficient de versatil pentru a permite utilizarea diferitelor protocoale de transport. Stivele standard folosesc HTTP ca protocol de transport, dar alte protocoale sunt, de asemenea, utilizabile (de exemplu, SMTP, RSS).
- SOAP este independent de platformă.
- SOAP este independentă de limbă.
- SOAP este simplu și extensibil.

Lista dezavantajelor este următoarea:

Din cauza formatului XML, SOAP poate fi considerabil mai lent decât tehnologiile middleware concurente.

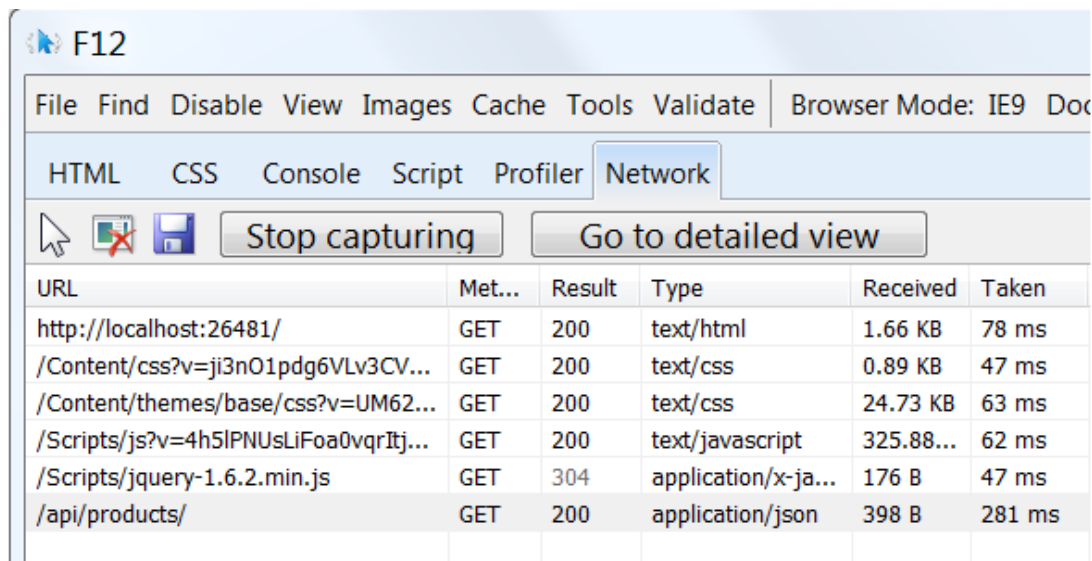
Atunci când se bazează pe HTTP ca protocol de transport și care nu utilizează WS-Adresare sau un ESB, rolurile părților interacționate sunt fixate. Numai o parte (clientul) poate utiliza serviciile celuilalt. Dezvoltatorii trebuie să utilizeze sondaje în loc de notificare în aceste cazuri obișnuite.

Cele mai multe utilizări ale HTTP ca protocol de transport se fac în ignoranța modului în care operația va fi modelată în HTTP. Acest lucru se întâmplă prin analogie cu modul în care diferite protocoale stau una peste cealaltă în stiva IP, dar analogia este imperfectă (deoarece protocoalele de aplicație folosite ca protocoale de transport nu sunt chiar protocoale de transport). Din acest motiv, nu există nici o modalitate de a ști dacă metoda utilizată este adecvată operației. Acest lucru face ca analiza bună a operației la nivelul protocolului de aplicație să fie problematică în cel mai bun caz cu rezultate care sunt suboptimale (dacă legarea POST este utilizată pentru o aplicație care în HTTP ar fi mai mult modelată în mod natural ca o operație GET).

2. Aplicatie Laborator

.Folosind F12 pentru a vizualiza solicitarea și răspunsul HTTP, la aplicatia din laboratorul trecut vom putea observa urmatoarele :

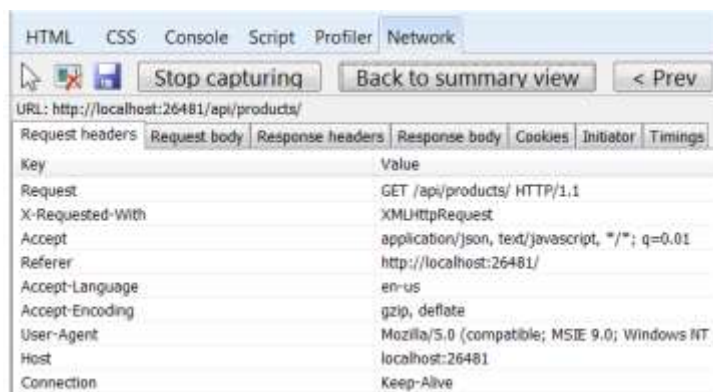
Când lucrați cu un serviciu HTTP, poate fi foarte util să vedeți solicitarea HTTP și să solicitați mesaje. Puteți face acest lucru utilizând instrumentele de dezvoltare F12 din Internet Explorer 9. Din Internet Explorer 9, apăsați F12 pentru a deschide instrumentele. Faceți clic pe fila Rețea și apăsați Start Captură. Acum reveniți la pagina web și apăsați F5 pentru a reîncărca pagina web. Internet Explorer va capta traficul HTTP între browser și serverul web. Rezumatul vizualizează întregul trafic din rețea pentru o pagină:



URL	Met...	Result	Type	Received	Taken
http://localhost:26481/	GET	200	text/html	1.66 KB	78 ms
/Content/css?v=ji3nO1pdg6VLv3CV...	GET	200	text/css	0.89 KB	47 ms
/Content/themes/base/css?v=UM62...	GET	200	text/css	24.73 KB	63 ms
/Scripts/js?v=4h5lPNUsLiFoa0vqrItj...	GET	200	text/javascript	325.88...	62 ms
/Scripts/jquery-1.6.2.min.js	GET	304	application/x-ja...	176 B	47 ms
/api/products/	GET	200	application/json	398 B	281 ms

Figura 4.1: Vizualizarea Serviciilor

Localizați intrarea pentru URI-ul relativ "api / products /". Selectați această intrare și faceți clic pe Mergeți la vizualizarea detaliată. În vizualizarea detaliată, există file pentru a vizualiza antetele și organismele de solicitare și de răspuns. De exemplu, dacă faceți clic pe fila Antet cerere, puteți vedea că clientul a solicitat "application / json" în antetul Accept.



Key	Value
Request	GET /api/products/ HTTP/1.1
X-Requested-With	XMLHttpRequest
Accept	application/json, text/javascript, */*; q=0.01
Referer	http://localhost:26481/
Accept-Language	en-us
Accept-Encoding	gzip, deflate
User-Agent	Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.0; ...)
Host	localhost:26481
Connection	Keep-Alive

Figura 4.2: Network

Dacă faceți clic pe tabul de răspuns, puteți vedea cum a fost serializată lista de produse JSON. Alte browsere au funcționalități similare. Un alt instrument util este Fiddler, un proxy de depanare web. Puteți utiliza Fiddler pentru a vizualiza traficul HTTP și pentru a compune și cereri HTTP, ceea ce vă oferă un control deplin asupra anteturilor HTTP din cerere.

Aplicatie 2 :

- Pas 1 : Select File -> New -> Web Site in Visual Studio, si apoi selectezi ASP.NET Web Service.
- Pas 2 : Un fișier de serviciu web numit Service.asmx și codul său din spatele fișierului, Service.cs este creat în directorul App_Code al proiectului.
- Pas 3 : Schimbați numele fișierelor la StockService.asmx și StockService.cs
- Pas 4 : Fișierul .asmx are pur și simplu o directivă WebService pe acesta:

```
<%@ WebService Language="C#" CodeBehind="~/App_Code/StockService.cs"
Class="StockService" %>
```

Pas 5 : Deschideți fișierul StockService.cs, codul generat în acesta este serviciul de bază Hello World. Codul implicit al serviciului web din spatele unui fișier arată după cum urmează:

```
using System;

using System.Collections;

using System.ComponentModel;

using System.Data;

using System.Linq;


using System.Web;

using System.Web.Services;

using System.Web.Services.Protocols;


using System.Xml.Linq;

namespace StockService
{
    // <summary>
```

```

// Summary description for Service1

// <summary>

[WebService(Namespace = "http://tempuri.org/")]
[WebServiceBinding(ConformsTo = WsiProfiles.BasicProfile1_1)]
[ToolboxItem(false)]

// To allow this Web Service to be called from script,
// using ASP.NET AJAX, uncomment the following line.
// [System.Web.Script.Services.ScriptService]

public class Service1 : System.Web.Services.WebService
{
    [WebMethod]

    public string HelloWorld()
    {
        return "Hello World";
    }
}
}

```

Pasul (6): Schimbați codul din spatele fișierului pentru a adăuga o matrice bidimensională de șiruri de caractere pentru simbolul stocului, nume și preț și două metode web pentru a obține informații despre stoc.

```

using System;

using System.Linq;

using System.Web;

using System.Web.Services;

```

```
using System.Web.Services.Protocols;

using System.Xml.Linq;

[WebService(Namespace = "http://tempuri.org/")]
[WebServiceBinding(ConformsTo = WsiProfiles.BasicProfile1_1)]

// To allow this Web Service to be called from script,
// using ASP.NET AJAX, uncomment the following line.
// [System.Web.Script.Services.ScriptService]

public class StockService : System.Web.Services.WebService
{
    public StockService () {
        //Uncomment the following if using designed components
        //InitializeComponent();
    }

    string[,] stocks =
    {
        {"RELIND", "Reliance Industries", "1060.15"},
        {"ICICI", "ICICI Bank", "911.55"},
        {"JSW", "JSW Steel", "1201.25"},
        {"WIPRO", "Wipro Limited", "1194.65"},
        {"SATYAM", "Satyam Computers", "91.10"}
    };

    [WebMethod]
    public string HelloWorld() {
        return "Hello World";
    }
}
```

```

[WebMethod]
public double GetPrice(string symbol)
{
    //it takes the symbol as parameter and returns price
    for (int i = 0; i < stocks.GetLength(0); i++)
    {
        if (String.Compare(symbol, stocks[i, 0], true) == 0)
            return Convert.ToDouble(stocks[i, 2]);
    }
    return 0;
}

[WebMethod]
public string GetName(string symbol)
{
    // It takes the symbol as parameter and
    // returns name of the stock
    for (int i = 0; i < stocks.GetLength(0); i++)
    {
        if (String.Compare(symbol, stocks[i, 0], true) == 0)
            return stocks[i, 1];
    }
    return "Stock Not Found";
}
}

```

Pasul (7): Rularea aplicației de serviciu web oferă o pagină de testare a serviciului web, care permite testarea metodelor de serviciu.

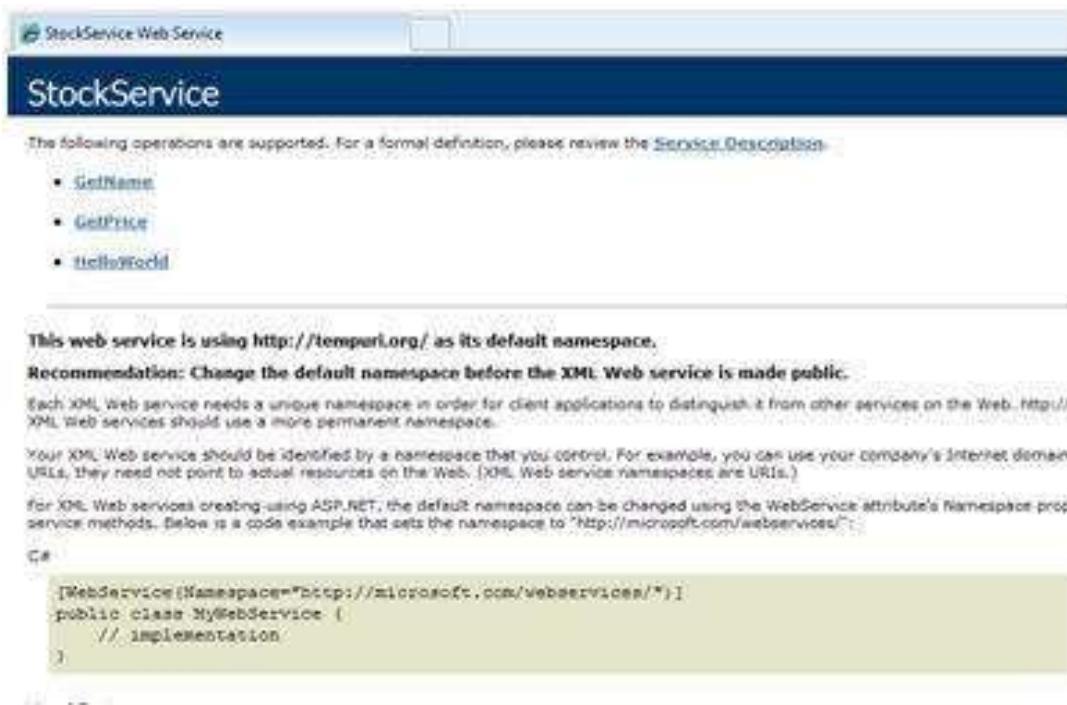


Figura 4.3 StockService

Pasul (8): faceți clic pe numele metodei și verificați dacă rulează corect.

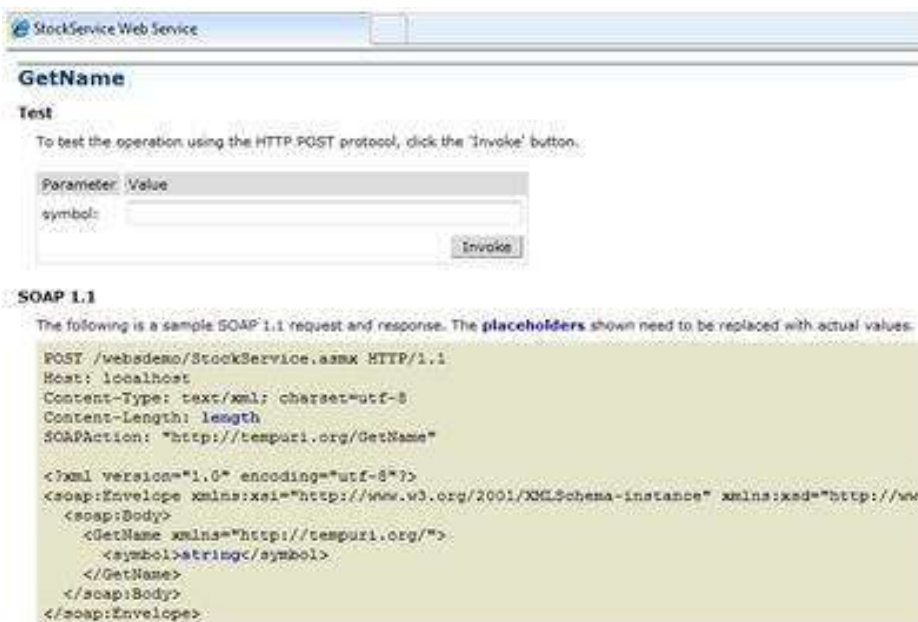


Figura 4.4 StockService - getname

Pasul (9): Pentru testarea metodei GetName, furnizați unul dintre simbolurile stoc, care sunt codate cu greu, returnează numele stocului



Pentru a utiliza serviciul web, creați un site web sub aceeași soluție. Acest lucru se poate face prin clic dreapta pe numele soluției din Solution Explorer. Pagina web care apelează serviciul web trebuie să aibă un control de etichetare pentru a afișa rezultatele returnate și două comenzi pentru butoane, unul pentru post înapoi și altul pentru a apela serviciul.

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Default.aspx.cs"
Inherits="wsclient._Default" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml" >

    <head runat="server">

        <title>

            Untitled Page

        </title>

    </head>

    <body>

        <form id="form1" runat="server">

            <div>

                <h3>Using the Stock Service</h3>

                <br /> <br />

                <asp:Label ID="lblmessage" runat="server"></asp:Label>

                <br /> <br />

                <asp:Button ID="btnpostback" runat="server" onclick="Button1_Click"
Text="Post Back" style="width:132px" />

            </div>

        </form>

    </body>

</html>
```

```

        <asp:Button ID="btnservice" runat="server" onclick="btnservice_Click"
Text="Get Stock" style="width:99px" />

    </div>

</form>

</body>

</html>

```

Codul pentru aplicația web este după cum urmează:

```

using System;
using System.Collections;
using System.Configuration;
using System.Data;
using System.Linq;
using System.Web;
using System.Web.Security;
using System.Web.UI;
using System.Web.UI.HtmlControls;
using System.Web.UI.WebControls;
using System.Web.UI.WebControls.WebParts;
using System.Xml.Linq;
//this is the proxy
using localhost;

namespace wsclient
{
    public partial class _Default : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)

```

```

{
    if (!IsPostBack)
    {
        lblmessage.Text = "First Loading Time: " + DateTime.Now.ToLongTimeString();
    }
    else
    {
        lblmessage.Text = "PostBack at: " + DateTime.Now.ToLongTimeString();
    }
}

protected void btnservice_Click(object sender, EventArgs e)
{
    StockService proxy = new StockService();

    lblmessage.Text = String.Format("Current SATYAM Price:{0}",
    proxy.GetPrice("SATYAM").ToString());
}
}
}

```

Un proxy este un stand-in pentru codurile de serviciu web. Înainte de a utiliza serviciul web, trebuie creat un proxy. Proxy-ul este înregistrat la aplicația client. Apoi, aplicația client face apelurile către serviciul web pe măsură ce utilizează o metodă locală. Proxy-ul primește apelurile, îl înfășoară în format corespunzător și îl trimite ca cerere SOAP către server. SOAP înseamnă Protocolul de acces simplu de obiect. Acest protocol este utilizat pentru schimbul de date privind serviciile web. Când serverul returnează pachetul SOAP clientului, proxy-ul decodifică totul și îl prezintă aplicației client. Înainte de a apela serviciul web utilizând `btnservice_Click`, trebuie adăugată o referință web la aplicație. Aceasta creează transparent o clasă proxy, care este utilizată de evenimentul `btnservice_Click`.

```
protected void btnservice_Click(object sender, EventArgs e)
```

```

{
    StockService proxy = new StockService();
    lblmessage.Text = String.Format("Current SATYAM Price: {0}",
    proxy.GetPrice("SATYAM").ToString());
}

```

Pasul (1): faceți clic dreapta pe intrarea aplicației web din Solution Explorer și faceți clic pe "Add Web Reference".

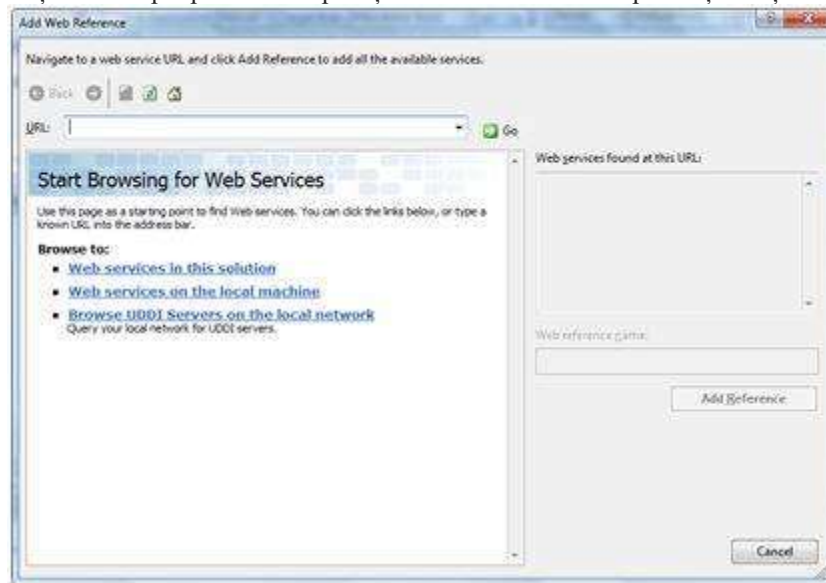


Figura 4.5 Referința Web

Pasul (2): Selectați "Servicii Web în această soluție". Aceasta returnează referința StockService.

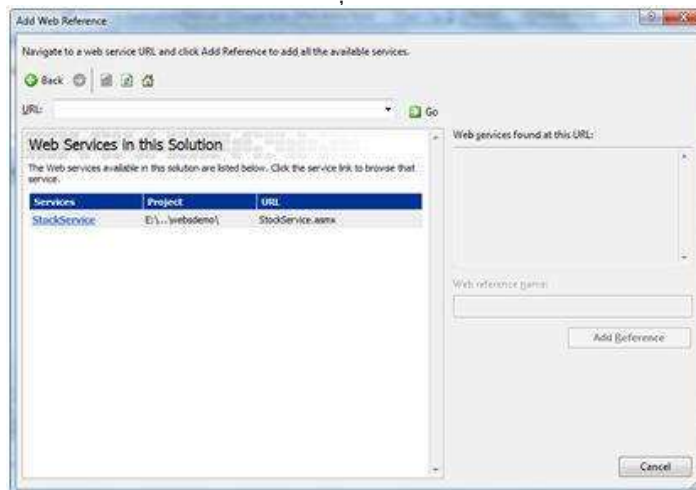


Figura 4.6: Referința StockService

Pasul (3): Dând clic pe serviciu, se deschide pagina de testare. Implicit proxy-ul creat se numește 'localhost', îl poți redenumi. Dați clic pe "Adăugați o referință" pentru a adăuga proxy-ul la aplicația client.

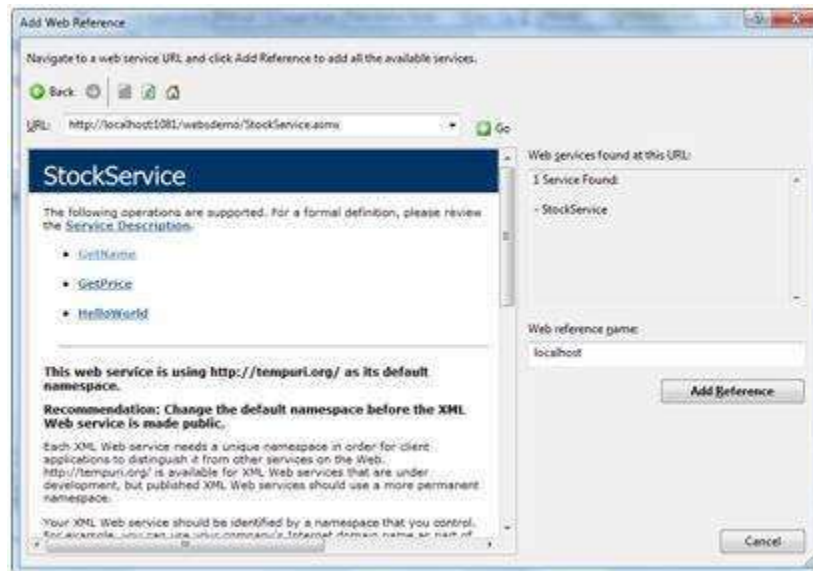


Figura 4.7: Proxy Client

3. Cerinte Laborator :

3.1. Implementati serviciul web explicat mai sus.

4. Bibliografie

1. <https://searchmicroservices.techtarget.com/tutorial/Simple-Object-Access-Protocol-SOAP-Tutorial>
2. <https://docs.microsoft.com/en-us/aspnet/web-api/overview/getting-started-with-aspnet-web-api/tutorial-your-first-web-api>
3. https://www.tutorialspoint.com/asp.net/asp.net_web_services.htm
- 4.

